

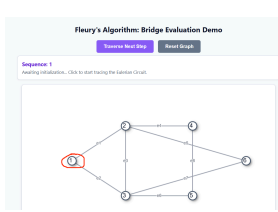
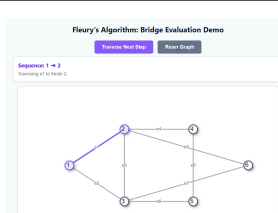
Graph Theory Group Assignment 2 Report

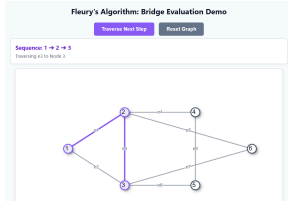
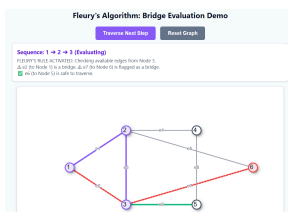
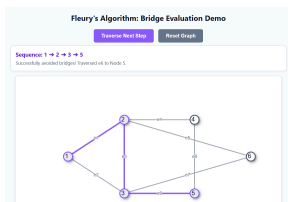
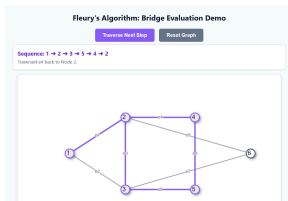
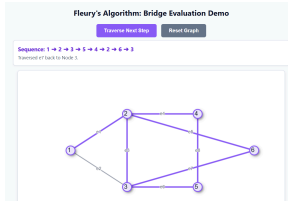
GROUP 8

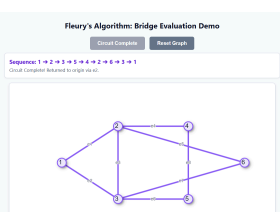
NRP	Name
5025251260	Aqilah Ibrahim
5025251161	Rizqi Arya Kuskhilbyano
5025251009	Athar Rozy Rasyidan
5025251017	Wafi Fawwaz Sutisna

1. Fleury's Implementation

Fleury's Algorithm finds an Eulerian circuit or path by traversing edges one at a time — starting anywhere for a circuit, or at an odd-degree vertex for a path — always avoiding a bridge unless forced to cross one, since that could strand the walk away from remaining edges. Each used edge is removed, and the process repeats until none are left. It's intuitive by hand, but checking for bridges every step makes it $O(E^2)$, slower than Hierholzer's $O(E)$ approach used in practice.

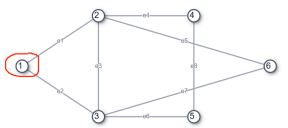
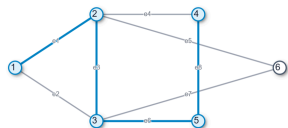
STEP	EXPLANATION	STATE	VISUAL
Initial	Start at Node 1	1	
1	Traverse e_1	$1 \rightarrow 2$	

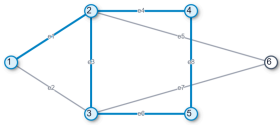
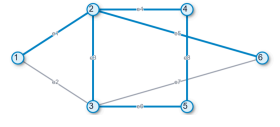
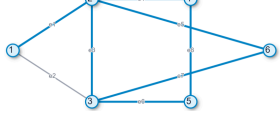
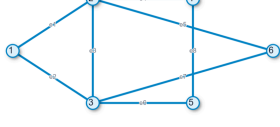
2	From Node 2, selects an arbitrary available edge. Traverses e3 to Node 3.	$1 \rightarrow 2 \rightarrow 3$	
3 (eval)	Evaluates remaining incident edges at Node 3 (e2, e6, e7). Identifies that cutting e2 or e7 would isolate unvisited nodes, making them bridges. Identifies e6 as a non-bridge.	$1 \rightarrow 2 \rightarrow 3$	
4	Applies Fleury's strict rule: avoids bridges when other options exist. Traverses the safe edge e6 to Node 5.	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5$	
5	Node 5 has only one untraversed incident edge. Traverses e8 to Node 4.	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4$	
6	Node 4 has only one untraversed incident edge. Traverses e4 to Node 2.	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2$	
7	From Node 2, the only remaining untraversed edge is e5 . Traverses to Node 6.	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 6$	
8	Node 6 has only one remaining edge (e7). Because it is the only option, the algorithm is permitted to cross this bridge. Traverses to Node 3.	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 6 \rightarrow 3$	

9	Node 3 has one final remaining edge (e2). Traverses back to Node 1. Circuit complete.	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow$ $4 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow$ 1	
---	--	---	---

2. Hierholzer's Implementation

The second approach utilizes a recursive Depth-First Search (DFS) for cycle tracing. Vertices are appended to the circuit in post-order during the backtracking phase of the DFS, ensuring the final path is valid. Our implementation in hierholzer.c explicitly manages memory for the large test cases presented in CSES 1691. Since the recursion depth can reach up to 200,000 edges, we utilized pthread to expand the stack size, preventing stack overflow. This algorithm achieves an optimal $O(V + E)$ complexity and receives an "Accepted" status on CSES.

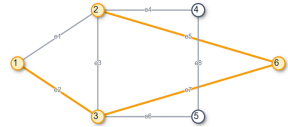
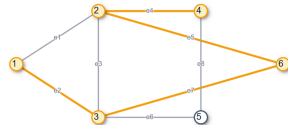
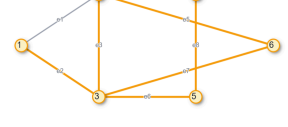
STEP	EXPLANATION	STATE	VISUAL
Initial	The algorithm initializes at the designated starting vertex, Node 1.	1	
1	From Node 1, traverse edge e1 to reach Node 2.	$1 \rightarrow 2$	
2	From Node 2, traverse edge e3 to reach Node 3.	$1 \rightarrow 2 \rightarrow 3$	
3	From Node 3, traverse edge e6 to reach Node 5.	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5$	
4	From Node 5, traverse edge e8 to reach Node 4.	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4$	

5	From Node 4, traverse edge e4 to return to Node 2.	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2$	
6	From Node 2, traverse edge e5 to reach Node 6.	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 6$	
7	From Node 6, traverse edge e7 to return to Node 3.	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 6 \rightarrow 3$	
8	Traverse the final available edge, e2 , to return to the origin Node 1.	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 1$	

3. Cycle Decomposition

The iterative sub-cycle splicing strategy implemented in `decomposition.c` effectively eliminates the recursion risks found in the recursive variant. It maintains a main tour using a linked list structure and uses a secondary structure (like a queue or stack) to track vertices with remaining unused incident edges. When a vertex with unused edges is found along the current tour, a new closed sub-cycle is traced and spliced directly into the main tour. This iterative approach maintains $O(V + E)$ linear time complexity and is highly robust for massive graphs, passing all CSES 1691 test cases without requiring stack adjustments.

STEP	EXPLANATION	STATE	VISUAL
initial	The cycle decomposition trace initializes at the starting vertex, Node 1.	1	
1	From Node 1, traverse edge e2 to reach Node 3.	$1 \rightarrow 3$	

2	From Node 3, traverse edge e7 to reach Node 6.	$1 \rightarrow 3 \rightarrow 6$	
3	From Node 6, traverse edge e5 to reach Node 2.	$1 \rightarrow 3 \rightarrow 6 \rightarrow 2$	
4	From Node 2, traverse edge e4 to reach Node 4.	$1 \rightarrow 3 \rightarrow 6 \rightarrow 2 \rightarrow 4$	
5	From Node 4, traverse edge e8 to reach Node 5.	$1 \rightarrow 3 \rightarrow 6 \rightarrow 2 \rightarrow 4 \rightarrow 5$	
6	From Node 5, traverse edge e6 to return to Node 3.	$1 \rightarrow 3 \rightarrow 6 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3$	
7	From Node 3, traverse edge e3 to return to Node 2.	$1 \rightarrow 3 \rightarrow 6 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 2$	
8	Traverse the final available edge, e1 , to return to the origin Node 1.	$1 \rightarrow 3 \rightarrow 6 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 2 \rightarrow 1$	

4. Complexity Analysis and Best Performing Algorithm

Algorithm	Typical implementation	Time complexity
Fleury	Repeatedly test whether candidate edge is a bridge	$O(E^2)$
Hierholzer (Recursive)	Build cycles/trails via post-order DFS and backtrack	$O(E) / O(V + E)$
Iterative Cycle Decomposition	Build cycles/trails and splice them together iteratively	$O(E) / O(V + E)$

Conclusion: Based on our theoretical and empirical evaluation, **Iterative Cycle Decomposition** is the optimal algorithm for competitive programming and real-world implementations. It preserves the optimal linear time complexity $O(V + E)$ of Hierholzer's approach while eliminating the $O(E^2)$ bridge-checking bottleneck of Fleury's algorithm and completely avoiding call-stack overflow risks on large inputs without needing pthread stack adjustments.